

Programming in Modern C++: Assignment Week 9

Total Marks : 25

Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur, Kharagpur – 721302
partha.p.das@gmail.com

September 13, 2025

Question 1

Consider the following program.

[MCQ, Marks 2]

```
#include<cstdio>

int main(){
    int num = 90;
    std::printf(_____, num, num, num, num);    //LINE-1
    return 0;
}
```

Identify the correct option to fill in the blank at LINE-1 such that the output becomes 90 132 5a Z.

- a) "%d %o %x %c"
- b) "%d %x %o %c"
- c) "%d %0x %o %s"
- d) "%i %0x %o %s"

Answer: a)

Explanation:

To print the value of `num` in decimal format, we use `%d`.

To print the value of `num` in octal format, we use `%o`.

To print the value of `num` in hexadecimal format, we use `%x`.

To print the value of `num` in char format, we use `%c`.

Question 2

Consider the following code segment.

[MCQ, Marks 2]

```
#include <stdio>

int main() {
    FILE *fp;
    if ((fp = std::fopen("records.txt", "r")) == NULL)
        return -1;

    int c, count = 0;
    while ((c = std::fgetc(fp)) != EOF) {
        if (!((c >= 'A' && c <= 'Z') ||
            (c >= 'a' && c <= 'z') ||
            (c >= '0' && c <= '9'))
            count++;
    }

    std::printf("%d", count);
    fclose(fp);
    return 0;
}
```

Choose the correct option regarding the program.

- a) It prints the number of alphanumeric characters in `records.txt`.
- b) It prints the total number of characters in `records.txt`.
- c) It prints the number of special characters and whitespace in `records.txt`.
- d) It prints the number of words in `records.txt`.

Answer: c)

Explanation:

The program:

- Reads every character using `std::fgetc()`.
- Uses a logical negation to skip alphabets and digits.
- Increments `count` only for characters that are **not** in A-Z, a-z, or 0-9.

Thus, it prints the number of special characters and whitespace (spaces, tabs, newlines, punctuation, etc.) present in `records.txt`.

Question 3

Consider the following code segment.

[MCQ, Marks 2]

```
#include <iostream>
#include <iomanip>

int main () {
    double num = 22.0 / 7;
    std::cout << std::fixed << std::setprecision(2);
    std::cout << std::setfill('*') << std::setw(10) << num;
    return 0;
}
```

What will be the output?

- a) 3.14*****
- b) *****3.14
- c) ***3.142857
- d) 3.142857***

Answer: b)

Explanation:

- The statement `std::fixed` ensures the number is printed in fixed-point notation instead of scientific.
- The statement `std::setprecision(2)` prints exactly 2 digits after the decimal point, so `num` becomes 3.14.
- The statement `std::setw(10)` sets the total width of the output to 10.
- The statement `std::setfill('*')` fills the unused width with asterisks on the left by default.

Thus, `num` is right-aligned in a field of width 10, resulting in the output `*****3.14`.

Question 4

Consider the following code segment (in C++14).

[MCQ, Marks 1]

```
#include <iostream>
#include <fstream>

int main () {
    std::ifstream fin("data.txt");
    if (_____) { //LINE-1
        std::cout << "file does not exist";
    }
    else {
        std::cout << "file exists";
        fin.close();
    }
    return 0;
}
```

Which of the following options correctly fills the blank at LINE-1 to check if the file does not exist?

- a) `fin.is_open()`
- b) `!fin.is_open()`
- c) `fin.open()`
- d) `!fin.exits()`

Answer: b)

Explanation:

The correct way to check if a file could not be opened is by using `!fin.is_open()`.

Question 5

Consider a scenario where a file `report.txt` contains 200 characters. You need to move the file pointer 30 bytes forward from the current position in the file. Which of the following statements correctly achieves this? *[MCQ, Marks 2]*

- a) `fseek(fp, 30, SEEK_SET);`
- b) `fseek(fp, 30, SEEK_CUR);`
- c) `fseek(fp, 30, SEEK_END);`
- d) `fseek(fp, -30, SEEK_CUR);`

Answer: b)

Explanation:

- `SEEK_SET` moves the file pointer relative to the start of the file.
- `SEEK_CUR` moves the file pointer relative to the current position.
- `SEEK_END` moves the file pointer relative to the end of the file.
- Since we want to move 30 bytes forward from the current position, the correct option is `fseek(fp, 30, SEEK_CUR)`.

Question 6

Consider the following program (in C++11) to compute the *total discounted price* of items stored in a vector `price` and corresponding discount percentages stored in a list `discount`.
[MCQ, Marks 2]

```
#include <iostream>
#include <list>
#include <vector>
#include <numeric>

double add(double a, double b){ return a + b; }
double applyDiscount(double price, double discount){
    return price - (price * discount / 100);
}

int main() {
    std::vector<double> price { 100, 200, 300, 400, 500 };
    std::list<double> discount { 10, 20, 10, 5, 15 };

    double total = inner_product(_____);    //LINE-1
    std::cout << total;
    return 0;
}
```

Choose the correct option to fill in the blank at LINE-1 so that the program outputs the *total discounted price* as 1325.

- a) `price.begin(), price.end(), discount.begin(), 0.0, add, applyDiscount`
- b) `discount.begin(), discount.end(), price.begin(), 0.0, add, applyDiscount`
- c) `price.begin(), price.end(), discount.begin(), 0.0, applyDiscount, add`
- d) `discount.begin(), discount.end(), price.begin(), 0.0, applyDiscount, add`

Answer: a)

Explanation:

The `inner_product` function combines two ranges element-wise using the second binary operation, and accumulates the results using the first binary operation. Here:

- `applyDiscount(price, discount) →` computes the price after discount.
- `add(total, discountedPrice) →` accumulates the total.

Thus, whether we iterate over `price` first or `discount` first, options a) and b) produce the same correct output.

Question 7

Consider the following code segment (in C++11).

[MCQ, Marks 2]

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <string>

class Employee {
public:
    Employee(int id, std::string name, double salary)
        : id_(id), name_(name), salary_(salary) {}
    int get_id() const { return id_; }
    std::string get_name() const { return name_; }
    double get_salary() const { return salary_; }
private:
    int id_;
    std::string name_;
    double salary_;
};

struct comparator {
    bool operator()(const Employee &e1, const Employee &e2) const {
        if(e1.get_salary() == e2.get_salary())
            return e1.get_name() < e2.get_name();
        return e1.get_salary() > e2.get_salary();
    }
};

int main() {
    Employee e[] = {
        Employee(101, "Alice", 75000),
        Employee(104, "Bob", 80000),
        Employee(102, "Charlie", 80000),
        Employee(103, "David", 70000)
    };

    std::vector<Employee> e_list(e, e + sizeof(e) / sizeof(*e));
    std::sort(e_list.begin(), e_list.end(), comparator());

    for(auto it = e_list.begin(); it != e_list.end(); ++it)
        std::cout << it->get_id() << " : " << it->get_name()
            << " : " << it->get_salary() << std::endl;

    return 0;
}
```

What will be the output?

- a) 104 : Bob : 80000
- 102 : Charlie : 80000
- 101 : Alice : 75000
- 103 : David : 70000

b) 102 : Charlie : 80000
104 : Bob : 80000
101 : Alice : 75000
103 : David : 70000

c) 103 : David : 70000
101 : Alice : 75000
102 : Charlie : 80000
104 : Bob : 80000

d) 104 : Bob : 80000
102 : Charlie : 80000
103 : David : 70000
101 : Alice : 75000

Answer: a)

Explanation: According to the functor `comparator`, the vector `e_list` is first sorted by `salary` in descending order. If salaries are equal, it is sorted by `name` in ascending order. Thus, employees with the highest salaries (80000) come first, sorted by name (Bob before Charlie), followed by the rest. Therefore, the correct option is a).

Question 8

Consider the following code segment.

[MSQ, Marks 2]

```
#include<iostream>
#include<string>
#include<vector>

template<class Itr>
int findLongest(Itr first, Itr last, std::string& longest) {
    int pos = 0, maxpos = 0;
    longest = *first++;
    while(first != last) {
        ++pos;
        if(first->length() > longest.length()) {
            longest = *first;
            maxpos = pos;
        }
        ++first;
    }
    return maxpos;
}

int main() {
    std::vector<std::string> names = {"red", "blue", "yellow", "green"};
    std::string lstr;
    -----; // LINE-1
    std::cout << pos << ", " << lstr;
    return 0;
}
```

Choose the appropriate options to fill in the blank at LINE-1 such that the program finds the longest string in the vector `names` and the output is: 2, yellow

- a) `findLongest(names.begin(), names.end(), lstr)`
- b) `int pos = findLongest(names.begin(), names.end(), lstr)`
- c) `std::string pos = findLongest(names.begin(), names.end(), lstr)`
- d) `int pos = findLongest(&names[0], &names[0] + names.size(), lstr)`

Answer: b), d)

Explanation:

The template function `findLongest` accepts:

- `first` – iterator or pointer to the beginning of the container.
- `last` – iterator or pointer to the end of the container.
- `longest` – reference variable to store the longest string.

It returns the index of the longest string in the container. Therefore, options **b)** and **d)** are correct.

Question 9

Consider the following code segment (in C++11).

[MCQ, Marks 2]

```
#include <iostream>
#include <vector>
#include <deque>
#include <algorithm>

int main() {
    std::deque<int> dq = { 10, 20, 30, 40, 50, 60 };
    std::vector<int> v(dq.size(), 0);

    std::deque<int>::iterator it1 = dq.begin();
    std::vector<int>::iterator it2 = v.begin();

    for (int i = 0; i < 3; i++) {    // LINE-1
        ++it1;
        ++it2;
    }

    std::copy(it1, dq.end(), it2);    // LINE-2

    for (it2 = v.begin(); it2 != v.end(); ++it2)
        std::cout << *it2 << " ";
    return 0;
}
```

What will be the output?

- a) 0 0 0 40 50 60
- b) 10 20 30 40 50 60
- c) 40 50 60 0 0 0
- d) 0 10 20 30 40 50

Answer: a)

Explanation:

The for loop at LINE-1 advances both `it1` and `it2` to the 4th position (0-based index) in their respective containers. Thus, the `copy` operation starting from `it1` copies the last three elements 40, 50, 60 from `dq` into `v` starting at index 3. Since the first three elements of `v` are not modified, they remain 0 0 0.

Programming Questions

Question 1

Question

Consider the following program (in C++11), which counts the frequency of each digit (0-9) in a given number. Fill in the blanks as per the instructions given below:

- Fill in the blank at LINE-1 with an appropriate condition to continue extracting digits from the number `n`.
- Fill in the blank at LINE-2 with an appropriate statement to update the frequency of the extracted digit.
- Fill in the blank at LINE-3 with an appropriate statement to iterate over the map `dFreq` and print it.

The program must satisfy the sample input and output.

[Fill in the Blanks, Marks 3]

```
#include <iostream>
#include <map>

std::map<int, int> findDigitFrequency(long long n){
    std::map<int, int> dFreq;
    while(_____) //LINE-1
    {
        int digit = n % 10;
        _____ //LINE-2
        n = n / 10;
    }
    return dFreq;
}

void print(std::map<int, int> dFreq){
    for (_____) //LINE-3
        std::cout << it->first << " -> " << it->second << std::endl;
}

int main(){
    long long num;
    std::cin >> num;
    std::map<int, int> dFreq = findDigitFrequency(num);
    print(dFreq);
    return 0;
}
```

Sample Test Cases

Public Test 1

Input: 120345

Output:

0 -> 1

1 -> 1

```
2 -> 1
3 -> 1
4 -> 1
5 -> 1
```

Public Test 2

Input: 100220

Output:

```
0 -> 3
1 -> 1
2 -> 2
```

Private Test

Input: 98766789

Output:

```
6 -> 2
7 -> 2
8 -> 2
9 -> 2
```

Answer:

LINE-1: `n > 0`

LINE-2: `dFreq[digit]++;`

LINE-3: `std::map<int, int>::iterator it = dFreq.begin(); it != dFreq.end(); ++it`

Explanation:

The program extracts digits one by one using modulus `n % 10`, increments their frequency in the map, and finally iterates through the sorted keys to display the counts.

At LINE-1, the condition to continue extracting digits from the number can be written as: `n > 0`.

At LINE-2, we can write a statement to update the frequency of the extracted digit as: `dFreq[digit]++;`

At LINE-3, we can write a statement to iterate over the map `dFreq` and print it as:

```
for (std::map<int, int>::iterator it = dFreq.begin(); it != dFreq.end(); ++it)
```

Question 2

Consider the following program that finds all elements in a list `li` which are greater than or equal to a given threshold value and prints them in the order they appear in the list.

- Fill in the blanks at LINE-1 with the appropriate template declaration.
- Fill in the blanks at LINE-2 with the correct usage of `find_if`.
- Fill in the blanks at LINE-3 with an appropriate loop to repeatedly call `find_if` and print all matching elements.

The program must satisfy the sample input and output.

Marks: 3

```
#include<iostream>
#include<list>
#include<algorithm>

template<typename T>
struct greaterThanOrEqual {
    T threshold_;
    greaterThanOrEqual(T threshold = 0) : threshold_(threshold) { }
    bool operator()(T x) { return x >= threshold_; }
};

----- // LINE-1
void printFromThreshold(std::list<T>& li, T threshold) {
    greaterThanOrEqual<T> pred(threshold);
    auto it = li.begin();
    while( (it = -----) != li.end() ) { // LINE-2
        std::cout << *it << " ";
        -----; // LINE-3
    }
}

int main(){
    std::list<int> li {30, 70, 10, 40, 20, 50, 60, 80};
    int threshold;
    std::cin >> threshold;
    printFromThreshold(li, threshold);
    return 0;
}
```

Public 1

Input: 40

Output: 70 40 50 60 80

Public 2

Input: 55

Output: 70 60 80

Private

Input: 75

Output: 80

Answer:

LINE-1: `template<typename T>`

LINE-2: `std::find_if(it, li.end(), pred)`

LINE-3: `++it`

Explanation:

- At LINE-1, we declare the function as a template so it works for any data type:
`template<typename T>`
- At LINE-2, we use `std::find_if` to find the next element in the list that satisfies the predicate:
`it = std::find_if(it, li.end(), pred);`
- The `while` loop ensures we repeatedly search for the next qualifying element until the end.
- At LINE-3, we increment the iterator so the next search continues from the element after the current match.
- If no element satisfies the predicate, `find_if` returns `li.end()`, and nothing is printed.

Question 3

Consider the following program which takes as input – the number of students, name and total marks of each student, followed by the type of sorting (1 for sort by marks in descending order and 2 for sort by name in ascending order). It prints the students' information in the corresponding sorted order. Complete the program with the following instructions.

- Fill the missing code segments at `code-segment-1` and `code-segment-2` with the appropriate overriding of function operator.

The program must satisfy the sample input and output.

Marks: 3

```
#include<iostream>
#include<string>
#include<algorithm>
#include<vector>
using namespace std;

class Student{
private:
    string name;
    int marks;
public:
    Student(string _name, int _marks) : name(_name), marks(_marks){}
    string getName() const{
        return name;
    }
    int getMarks() const{
        return marks;
    }
    friend ostream& operator<<(ostream& os, const Student& s);
};

ostream& operator<<(ostream& os, const Student& s){
    os << s.name << '-' << s.marks << endl;
    return os;
}

struct compareByMarksDesc{
    //code-segment-1
};

struct compareByNameAsc{
    //code-segment-2
};

int main() {
    int n;                //number of students
    cin >> n;
    vector<Student> students;
    for(int i = 0; i < n; i++){
        string na;
        int ma;
```

```

        cin >> na >> ma;    //student's name and marks
        Student s(na, ma);
        students.push_back(s);
    }
    int t;                //sort option
    cin >> t;
    if(t == 1)
        sort(students.begin(), students.end(), compareByMarksDesc());
    else if(t == 2)
        sort(students.begin(), students.end(), compareByNameAsc());
    for(vector<Student>::iterator p = students.begin(); p != students.end(); p++){
        cout << *p;
    }
    return 0;
}

```

Public 1

Input:
4
arjun 78
meera 95
kiran 88
sita 72
1
Output:
meera-95
kiran-88
arjun-78
sita-72

Public 2

Input:
5
raju 60
geeta 85
suresh 75
anita 80
bala 70
2
Output:
anita-80
bala-70
geeta-85
raju-60
suresh-75

Private

Input:
3
tina 92


```
reema 89
alok 95
1
Output:
alok-95
tina-92
reema-89
```

Answer:

```
code-segment-1:
bool operator()(const Student& s1, const Student& s2) const {
    return s1.getMarks() > s2.getMarks();
}
```

```
code-segment-2:
bool operator()(const Student& s1, const Student& s2) const {
    return s1.getName() < s2.getName();
}
```

Explanation:

At code-segment-1, the overloading of the function operator that sorts the students by marks in descending order can be done as follows:

```
bool operator()(const Student& s1, const Student& s2) const {
    return s1.getMarks() > s2.getMarks();
}
```

At code-segment-2, the overloading of the function operator that sorts the students by name in ascending order can be done as follows:

```
bool operator()(const Student& s1, const Student& s2) const {
    return s1.getName() < s2.getName();
}
```